🔍 Stock Flag Bot
Line-by-Line Code Walkthrough

Every core file, every important line explained in plain language

Companion to the Project Documentation & the AI/ML Study Guide · July 2026
Reading order follows the data's journey: market data → strategies → scoring → execution → automation → web server → browser extension

0. How To Read This Document

This walkthrough follows a single question — **"what happens, in code, from the moment you open a stock to the moment a trade is placed?"** — through every file it passes. Each section shows the real code (dark boxes) followed by a plain-language explanation of what each part does and *why* it's written that way.

The data flows through the codebase in this order, and so does this document:

1. `data.py` — fetch raw price candles from Yahoo Finance
2. `structure.py, zones.py, quant.py` — the strategy detectors that read those candles
3. `scoring.py` — combine all strategy votes into one BUY/SELL/HOLD flag
4. `engine.py` — the conductor that runs everything across timeframes
5. `virtual_broker.py, alpaca.py` — the "hands" that hold positions and place trades
6. `scanner.py` — the autonomous loop that decides buy/sell/hold + exits
7. `server.py, watchlists.py` — the web API and dashboards
8. `content.js, background.js` — the Chrome extension sidebar

Notation: code is shown exactly as it appears in the files. Line references like "`line 32`" match the source. Where a function is long, it's broken into chunks with explanation between them. Comments already in the code (green) are the author's; explanations in blue boxes are added here.

1. `data.py` — Getting Market Data

Everything starts here: without price candles, no strategy can run. This file's job is to turn a symbol like `NSE:RELIANCE` into clean tables of Open/High/Low/Close/Volume across four timeframes.

1.1 Corporate-proxy TLS handling (lines 7–24)

```
# Corporate proxies do TLS interception; trust the OS keychain so Python
# accepts the proxy's root cert (fixes bogus SSL/"rate limit" failures).
try:
    import truststore
    truststore.inject_into_ssl()
    _session = None  # default session now works through the proxy
except ImportError:
    try:
        from curl_cffi import requests as _cf_requests
        _session = _cf_requests.Session(impersonate="chrome")
    except Exception:
        _session = None
```

> **What/why:** On a company laptop, the network intercepts HTTPS traffic and re-signs it with a corporate certificate. Normal Python rejects that certificate. `truststore` tells Python to trust the operating system's certificate store (which already trusts the corporate cert), fixing errors that *look* like rate-limiting but are actually TLS failures. If that library isn't installed, it falls back to `curl_cffi` impersonating a real Chrome browser — which also helps avoid Yahoo blocking automated requests. This is defensive engineering: two independent ways to get a working network session.

1.2 Symbol mapping (the EXCHANGE_SUFFIX table + tradingview_to_yfinance)

```
EXCHANGE_SUFFIX = {
    "NSE": ".NS", "BSE": ".BO", "LSE": ".L", "HKEX": ".HK", "TSE": ".T", ...
}
```

> **What/why:** TradingView writes symbols as `EXCHANGE:TICKER` (e.g. `NSE:RELIANCE`), but Yahoo Finance uses suffixes (`RELIANCE.NS`). This dictionary is the translation table between the two systems. The function `tradingview_to_yfinance()` splits on the colon, looks up the exchange, and appends the right suffix — US exchanges (NASDAQ/NYSE) get no suffix, crypto becomes `BTC-USD` form, forex becomes `EURUSD=X`. This one mapping is what lets the same engine analyse Indian stocks, US stocks, and crypto without any other code changing.

1.3 The two-tier data fetch (lines 46–96)

```
def _fetch_chart(symbol, range_, interval):
    # Fetch OHLCV via Yahoo's public chart API with plain requests.
    r = requests.get(_CHART_URL.format(sym=symbol),
        params={"range": range_, "interval": interval, ...}, timeout=15)
    if r.status_code == 429:
        raise RuntimeError("rate limited")
    r.raise_for_status()
    result = (r.json().get("chart", {}).get("result") or [None])[0]
    ...
    df = pd.DataFrame({"Open": quote.get("open"), "High": quote.get("high"), ...},
        index=pd.to_datetime(result["timestamp"], unit="s", utc=True))
    return df.dropna(subset=["Close"])

def _history(symbol, range_, interval):
    """Chart API first, yfinance fallback."""
    try:
        df = _fetch_chart(symbol, range_, interval)
        if not df.empty: return df
    except RuntimeError: raise       # rate limit → let caller know
    except Exception: pass           # any other error → try yfinance
    return _ticker(symbol).history(period=range_, interval=interval)
```

> **What/why:** `_fetch_chart` calls Yahoo's raw chart JSON API directly with plain `requests` — this works through the corporate proxy, whereas the `yfinance` library's internal HTTP client does not. It parses the nested JSON into a clean pandas DataFrame with Open/High/Low/Close/Volume columns indexed by timestamp. `_history` wraps it in a **fallback pattern**: try the reliable chart API first; if it

> fails for any reason *other than* a rate limit, fall back to the yfinance library. A rate-limit error (HTTP 429) is deliberately re-raised so the caller can show an honest "try again later" message instead of a misleading "unknown symbol". This is the graceful-degradation principle applied to data fetching.

1.4 fetch_frames — building four timeframes (lines 58–79)

```
def fetch_frames(yf_symbol):
    frames = {}
    try: frames["1W"] = _clean(_history(yf_symbol, "5y", "1wk"))
    except Exception: frames["1W"] = pd.DataFrame()
    try: frames["1D"] = _clean(_history(yf_symbol, "2y", "1d"))
    except Exception: frames["1D"] = pd.DataFrame()
    try: h1 = _clean(_history(yf_symbol, "60d", "1h"))
    except Exception: h1 = pd.DataFrame()
    frames["1H"] = h1
    frames["4H"] = _resample_4h(h1)
    return frames
```

> **What/why:** The engine analyses four timeframes to gauge both the big picture (weekly) and the recent detail (hourly). This fetches 5 years of weekly, 2 years of daily, and 60 days of hourly candles. The 4-hour frame isn't fetched separately (Yahoo doesn't offer it free) — it's **resampled** from the 1-hour data by grouping every 4 bars (open=first, high=max, low=min, close=last, volume=sum). Every fetch is individually wrapped in try/except so that if one timeframe fails (common for intraday on some symbols), the others still work and the strategies that need them simply skip. This is why the daily flag still appears even when hourly data is unavailable.

1.5 resolve_symbol — trying candidates (lines 86–111)

```
def resolve_symbol(symbol):
    candidates = [tradingview_to_yfinance(symbol)]
    bare = symbol.split(":")[-1].strip().upper()
    for extra in (bare, bare + ".NS", bare + "-USD"):
        if extra not in candidates: candidates.append(extra)
    rate_limited = False
    for cand in candidates:
        try:
            df = _history(cand, "5d", "1d")
            if df is not None and not df.empty: return cand
        except Exception as e:
            if "rate limit" in str(e).lower(): rate_limited = True
    if rate_limited: raise DataSourceError(...)
    return None
```

> **What/why:** Sometimes the exact mapping doesn't have data — so this tries several candidate symbols in order (the mapped form, the bare ticker, the ticker with `.NS` for Indian stocks, with `-USD` for crypto) and returns the first that actually returns candles. It tracks whether any attempt hit a rate limit; if *all* candidates failed but at least one was rate-limited, it raises a specific `DataSourceError` so the UI can say "Yahoo is throttling, wait a moment" rather than "symbol not found." Distinguishing "no data" from "temporarily blocked" is what makes the error messages trustworthy.

2. `structure.py` — Market Structure

This file answers "is the stock trending up, down, or sideways, and did it just break a key level?" — the highest-weighted signals in the whole engine.

2.1 The Swing data class + find_swings (lines 9–43)

```
@dataclass
class Swing:
    kind: str    # "H" or "L"
    pos: int     # integer position in the frame
    price: float

def find_swings(df, lookback=3):
    highs, lows = df["High"].values, df["Low"].values
    n = len(df)
    raw = []
    for i in range(lookback, n - lookback):
        window_h = highs[i - lookback: i + lookback + 1]
        window_l = lows[i - lookback: i + lookback + 1]
        if highs[i] == window_h.max():
            raw.append(Swing("H", i, float(highs[i])))
        if lows[i] == window_l.min():
            raw.append(Swing("L", i, float(lows[i])))
    raw.sort(key=lambda s: s.pos)
    # collapse consecutive same-kind swings to the more extreme one
    swings = []
    for s in raw:
        if swings and swings[-1].kind == s.kind:
            prev = swings[-1]
            better = (s.price > prev.price) if s.kind == "H" else (s.price < prev.price)
            if better: swings[-1] = s
        else: swings.append(s)
    return swings
```

> **What/why:** A "swing high" is a peak — a bar whose high is the highest within a window of ±3 bars around it (this is called a **fractal**). Same idea for swing lows (valleys). The loop walks every bar and records those that are local extremes. The second loop **collapses runs**: if two swing highs appear in a row with no low between them, it keeps only the higher one — this forces the result to strictly alternate High, Low, High, Low, which is what trend analysis needs. The `.values` calls convert pandas columns to raw numpy arrays for speed, since this runs on thousands of bars.

2.2 classify_trend (lines 46–60)

```
def classify_trend(swings):
    hs = [s for s in swings if s.kind == "H"][-3:]
    ls = [s for s in swings if s.kind == "L"][-3:]
    if len(hs) < 2 or len(ls) < 2: return "range"
    higher_highs = hs[-1].price > hs[-2].price
    higher_lows  = ls[-1].price > ls[-2].price
    lower_highs  = hs[-1].price < hs[-2].price
    lower_lows   = ls[-1].price < ls[-2].price
    if higher_highs and higher_lows: return "up"
    if lower_highs and lower_lows: return "down"
    return "range"
```

> **What/why:** This is the textbook definition of trend, coded directly. An uptrend makes **higher highs and higher lows** (each peak and each valley above the last — like climbing stairs). A downtrend makes lower highs and lower lows. Anything else is a sideways range. It only looks at the last 2–3 swings of each kind because trend is about *recent* structure, not ancient history. If there aren't enough swings yet, it safely returns "range". This single function feeds the market-structure signal (weight 1.5, the highest) and the multi-timeframe alignment signal.

2.3 detect_break — BOS and CHOCH (lines 63–92)

```
def detect_break(df, swings, trend):
    close = df["Close"].values
    n = len(df)
    last_high = next((s for s in reversed(swings) if s.kind == "H"), None)
    last_low  = next((s for s in reversed(swings) if s.kind == "L"), None)
    events = []
    if last_high is not None:
        broke = [i for i in range(last_high.pos + 1, n) if close[i] > last_high.price]
        if broke and n - broke[0] <= 10:
            kind = "BOS" if trend == "up" else "CHOCH"
            events.append({"event": kind, "direction": "bullish", ...})
    # (mirror logic for last_low → bearish)
    return min(events, key=lambda e: e["bars_ago"]) if events else None
```

> **What/why:** This detects the two most important events in "smart money" trading. It finds the most recent swing high and low, then checks if price has **closed beyond** either one recently (within 10 bars). The clever part is the labeling: if price breaks the swing high *while already in an uptrend*, that's a **BOS** (Break of Structure — the trend continues). But if it breaks a level *against* the prevailing trend, that's a **CHOCH** (Change of Character — a possible reversal, which is why CHOCH gets higher strength in the engine). If both a high and low broke, it returns the more recent event (`min` by `bars_ago`). The 10-bar recency filter ensures we only report breaks that are still "news," not ancient history.

3. `zones.py` — Smart-Money Concepts

3.1 find_zones — supply/demand (lines 11–54)

```
def find_zones(df, impulse_atr_mult=2.0, max_zones=6):
    a = atr(df).values                          # ATR = typical bar size
    close, high, low, op = ...values
    zones = []
    for i in range(5, n - 3):
        move = close[i + 3] - close[i]
        if a[i] and abs(move) >= impulse_atr_mult * a[i]:   # sharp departure?
            kind = "demand" if move > 0 else "supply"
            top = max(high[i], op[i]); bottom = min(low[i], op[i])
            mitigated = False
            for j in range(i + 4, n):     # has price since closed through it?
                if kind == "demand" and close[j] < bottom: mitigated = True; break
                if kind == "supply" and close[j] > top:    mitigated = True; break
            if not mitigated:
                zones.append({"kind": kind, "top": top, "bottom": bottom, "pos": i})
    # dedupe overlapping zones, keep most recent
    return merged[:max_zones]
```

> **What/why:** The theory: when price **rockets away** from an area, big institutions were filling large orders there, and price often reacts when it returns. The code detects a "rocket" as a move of at least 2× ATR (the stock's typical bar size) over 3 bars. The candles right before that move become a "zone" — a demand zone if price shot up (expect a bounce when revisited), a supply zone if it dropped. Crucially, it checks whether the zone is still **"unmitigated"**: if price has since closed clean through it, the zone is spent and dropped. Only fresh, untested zones are kept. ATR is the key: it makes "sharp move" relative to each stock's own volatility instead of a fixed percentage.

3.2 detect_liquidity_sweep — stop hunts (lines 99–117)

```
def detect_liquidity_sweep(df, swings, lookback=8):
    recent_highs = [s for s in swings if s.kind == "H" and s.pos < n - lookback]
    recent_lows  = [s for s in swings if s.kind == "L" and s.pos < n - lookback]
    for i in range(max(0, n - lookback), n):
        for s in recent_lows[-3:]:
            if low[i] < s.price and close[i] > s.price:      # wick below, close back above
                return {"direction": "bullish", "level": s.price, ...}
        for s in recent_highs[-3:]:
            if high[i] > s.price and close[i] < s.price:
                return {"direction": "bearish", "level": s.price, ...}
    return None
```

> **What/why:** Many traders put stop-loss orders just beyond obvious highs/lows. A "liquidity sweep" (stop hunt) is when price briefly **spikes past** such a level — triggering those stops, letting big players fill — then **snaps back**. The code detects exactly this: a bar whose *wick* pierces a prior swing low but whose *close* comes back above it → bullish (the sellers who were stopped out are exhausted). Mirror for highs → bearish. The `close[i] > s.price` after `low[i] < s.price` is the whole trick: pierced but rejected. This is one of the most respected reversal signals in modern price-action trading.

4. `quant.py` — Systematic Signals

4.1 mean_reversion_signal — with the volatility filter (lines 10–27)

```
def mean_reversion_signal(df):
    z = float(zscore(df["Close"]).iloc[-1])       # how many std-devs from average
    adx_val = float(adx(df)[0].iloc[-1] or 0)     # trend strength 0-100
    mid, upper, lower = bollinger(df["Close"])
    trending = adx_val >= 28   # strong trend: fade signals are unreliable
    if z <= -2.0 or price < float(lower.iloc[-1]):
        return {"direction": "bullish", "zscore": round(z, 2),
                "suppressed_by_trend": trending}
    if z >= 2.0 or price > float(upper.iloc[-1]):
        return {"direction": "bearish", ..., "suppressed_by_trend": trending}
    return None
```

What/why: Mean reversion bets that price stretched far from its average snaps back. The **z-score** measures "how many standard deviations is price from its 20-bar average" — a z of –2 means unusually cheap (buy), +2 unusually expensive (sell). Bollinger Bands are the same idea visually. The critical line is `trending = adx_val >= 28` : ADX measures trend strength, and in a *strong trend*, "stretched" prices keep stretching rather than reverting — so the engine doesn't delete the signal but flags it `suppressed_by_trend` , and later cuts its strength to a quarter (0.25 instead of 0.65). This **volatility filter** is what separates a professional mean- reversion implementation from a naive one that gets run over by trends.

4.2 trend_following_signal (lines 30–55)

```
def trend_following_signal(df):
    e20, e50 = ema(close, 20), ema(close, 50)
    s50 = sma(close, 50); s200 = sma(close, 200)
    bull = price > float(e20.iloc[-1]) > float(e50.iloc[-1])   # stacked averages
    golden = float(s50.iloc[-1]) > float(s200.iloc[-1])        # golden cross regime
    if bull and golden: direction = "bullish"
    elif bear and not golden: direction = "bearish"
    else: return {"direction": "neutral", ...}
    return {"direction": direction, "strength": "strong" if a >= 25 else "weak",
            "regime": "golden_cross" if golden else "death_cross", ...}
```

What/why: The simplest durable edge in markets: things in motion tend to stay in motion. This requires **two confirmations** to call bullish: (1) price above the 20-day EMA above the 50-day EMA — a "stacked" alignment showing short-term momentum leads long-term; and (2) the "golden cross" regime where the 50-day average is above the 200-day (the market's most-watched bull/bear line). Requiring both avoids false signals from a short-term bounce inside a bear market. ADX ≥ 25 upgrades strength from "weak" to "strong" (the trend has real force behind it). EMA weights recent prices more than SMA, so it reacts faster.

4.3 breakout_signal — Donchian + volume (lines 58–72)

```
def breakout_signal(df, channel=20):
    hi = float(df["High"].rolling(channel).max().shift(1).iloc[-1])
    lo = float(df["Low"].rolling(channel).min().shift(1).iloc[-1])
    vol = float(df["Volume"].iloc[-1])
    avg_vol = float(df["Volume"].rolling(channel).mean().iloc[-1])
    vol_confirm = avg_vol > 0 and vol > 1.3 * avg_vol
    if price > hi: return {"direction": "bullish", "level": hi, "volume_confirmed": vol_confirm}
    if price < lo: return {"direction": "bearish", "level": lo, ...}
    return None
```

What/why: A "Donchian breakout" fires when price closes above the highest high of the last 20 bars — escaping its recent range. The `.shift(1)` is important: it uses the highest high *excluding today*, so today's close breaking that prior range counts as new. Volume confirmation matters because breakouts on low volume often fail ("fakeouts"); the code checks whether today's volume is at least 1.3× the 20-bar average, and the engine gives volume-confirmed breakouts higher strength (0.8 vs 0.5). This mirrors the famous "Turtle Traders" system.

5. `scoring.py` — Turning 17 Votes Into One Flag

This is the heart of the decision. Every strategy produced a `Signal`; this file combines them all into BUY, SELL, or HOLD.

5.1 The Signal data class (lines 7–20)

```
@dataclass
class Signal:
    name: str          # e.g. "market_structure"
    timeframe: str     # "1W" | "1D" | "4H" | "1H" | "options"
    direction: int     # -1 bearish, 0 neutral, +1 bullish
    strength: float    # 0..1 — how clear is this signal
    weight: float      # relative importance of this strategy
    detail: str        # human-readable explanation
    data: dict = field(default_factory=dict)

    def to_dict(self):
        d = asdict(self)
        d["direction_label"] = {1: "bullish", 0: "neutral", -1: "bearish"}[self.direction]
        return d
```

> **What/why:** Every one of the ~30 signals (17 strategies × up to 4 timeframes) is this uniform object. Three numbers drive the math: **direction** (which way), **strength** (how confident this one detector is), and **weight** (how much we trust this *type* of signal in general). Separating strength from weight is a deliberate design: a mean-reversion signal can be very clear (high strength) but still count for little (low weight) because we trust the strategy less. The `detail` string is what the sidebar and Gemini show as the human reason. In ML terms, this is a feature with a value and an importance — exactly the shape a future ML model would consume.

5.2 aggregate — the weighted vote (lines 28–64)

```
def aggregate(signals):
    active = [s for s in signals if s.direction != 0]
    total_weight = sum(s.weight for s in signals) or 1.0
    score = sum(s.direction * s.strength * s.weight for s in signals) / total_weight
```

> **The core formula.** Each signal contributes `direction × strength × weight` — a bullish (+1), clear (0.9), important (1.5) signal contributes +1.35; a weak bearish one contributes a small negative. Sum them all and divide by the total weight to normalize into a score between –1 and +1. This is mathematically identical to a **weighted-average ensemble vote** in machine learning. Dividing by total weight means the scale is consistent regardless of how many signals fired.

```
    if score >= BUY_THRESHOLD:    flag = "BUY"      # BUY_THRESHOLD = 0.22
    elif score <= SELL_THRESHOLD: flag = "SELL"     # SELL_THRESHOLD = -0.22
    else:                         flag = "HOLD"
```

> **What/why:** The score is turned into a flag by two thresholds. The ±0.22 band around zero is the "not convincing enough" zone → HOLD. These thresholds are the main tuning knobs: raise them and the bot trades less often but with more conviction; lower them and it trades more. They were set by judgment and are exactly what the backtester exists to optimize with evidence.

```
    if active:
        bulls = sum(s.weight for s in active if s.direction > 0)
        bears = sum(s.weight for s in active if s.direction < 0)
        agreement = abs(bulls - bears) / (bulls + bears)
    confidence = round(min(1.0, 0.6 * min(1.0, abs(score) / 0.5) + 0.4 * agreement), 2)
```

> **What/why:** Confidence is *not* the same as the score. It blends two things: (1) the magnitude of the score (how far from neutral), weighted 60%, and (2) **agreement** — how one-sided the vote was, weighted 40%. Agreement is the key insight: 20 signals split 11-vs-9 is a coin-flip even if the score happens to clear the threshold, so its agreement (0.1) drags confidence down. A clean 18-vs-2 split gives high agreement. This is why the sidebar can show "BUY, but only 45% confident" — the flag and the certainty are two separate numbers.

```
    bullish = sorted((s for s in active if s.direction > 0),
                     key=lambda s: s.weight * s.strength, reverse=True)
    return {"flag": flag, "score": round(score, 3), "confidence": confidence,
            "bullish_reasons": [f"[{s.timeframe}] {s.detail}" for s in bullish[:6]],
            "bearish_reasons": [...], "signal_counts": {...}, "signals": [...]}
```

What/why: Finally it sorts the bullish and bearish signals by importance (`weight × strength`) and returns the top 6 of each as human-readable reasons — that's what you see in the sidebar and what Gemini cites. The full signal list is also returned for the LLM. This return dict is the single source of truth every other part of the system consumes.

6. `engine.py` — The Conductor

6.1 _analyze_frame — running all strategies on one timeframe (lines 25–130)

```
def _analyze_frame(tf, df):
    signals = []
    if df is None or len(df) < 40: return signals    # need enough bars
    m = TF_MULT.get(tf, 1.0)                           # timeframe weight multiplier
    swings = find_swings(df)
    trend = classify_trend(swings)

    # 1. Market structure (HH/HL vs LH/LL)
    tdir = 1 if trend == "up" else -1 if trend == "down" else 0
    signals.append(Signal("market_structure", tf, tdir, 0.8 if tdir else 0.2, 1.5 * m, ...))

    # 2. BOS / CHOCH
    brk = detect_break(df, swings, trend)
    if brk:
        strength = 0.9 if brk["event"] == "CHOCH" else 0.7
        signals.append(Signal("bos_choch", tf, _DIR[brk["direction"]], strength, 1.2 * m, ...))
    # ... strategies 3–13 follow the same pattern ...
    return signals
```

What/why: This runs all 13 price-based strategies on a *single* timeframe and collects their signals. The pattern repeats: call the detector, and if it returns something, wrap it in a `Signal` with a direction, strength, and weight. Notice `1.5 * m`: the base weight (1.5 for market structure) is multiplied by `m`, the timeframe multiplier (`TF_MULT = {"1W": 1.3, "1D": 1.0, "4H": 0.8, "1H": 0.6}`). So the **same signal on the weekly chart counts more** than on the hourly — higher timeframes are more reliable. The `len(df) < 40` guard means strategies simply don't run if there isn't enough history, which is how the engine tolerates missing intraday data gracefully.

6.2 _mtf_alignment — multi-timeframe confluence (lines 133–153)

```
def _mtf_alignment(frames):
    trends = {}
    for tf in ("1W", "1D", "4H"):
        df = frames.get(tf)
        if df is not None and len(df) >= 40:
            trends[tf] = classify_trend(find_swings(df))
    ups = sum(1 for t in trends.values() if t == "up")
    downs = sum(1 for t in trends.values() if t == "down")
    if ups == len(trends):
        return Signal("mtf_alignment", "1D", 1, 0.9, 1.5, "all timeframes aligned up", ...)
    # ... partial agreement → weaker signal ...
```

What/why: This is a "meta" signal that checks whether the weekly, daily, and 4-hour trends **agree**. When all three point up, it's a powerful confluence (strength 0.9, weight 1.5). When they conflict, it returns a weak or neutral signal. Trading a setup that all timeframes agree on is far more reliable than one where the hourly says up but the weekly says down — this signal encodes that professional principle as a single high-weight vote.

6.3 analyze_ticker — the public entry point (lines 171–218)

```
def analyze_ticker(symbol, include_options=True):
    try: yf_symbol = resolve_symbol(symbol)
    except DataSourceError as e: return {"symbol": symbol, "error": str(e)}
    if yf_symbol is None: return {..., "error": "Could not resolve"}

    frames = fetch_frames(yf_symbol)
    if daily is None or daily.empty: return {..., "error": "No price data"}

    signals = []
    for tf in ("1W", "1D", "4H", "1H"):
        signals.extend(_analyze_frame(tf, frames.get(tf)))
    mtf = _mtf_alignment(frames)
    if mtf: signals.append(mtf)
    if opt: signals.append(Signal("options_sentiment", ...))

    result = scoring.aggregate(signals)
    result.update({"symbol": symbol.upper(), "yf_symbol": yf_symbol,
                   "price": round(price, 4), "change_pct": ..., ...})
    return result
```

What/why: This is the one function the whole rest of the system calls. It orchestrates the entire pipeline: resolve the symbol → fetch four timeframes → run every strategy on each → add the multi-timeframe and options signals → aggregate into a flag → attach price info. Every failure mode returns a clean `{"error": ...}` dict instead of crashing, so callers (the web server, the scanner) can always handle it. The `include_options` flag lets the scanner skip the slow options fetch when it doesn't need it. This function is also exposed to the LLM agents as the `get_technical_flag` tool.

7. `virtual_broker.py` — The Indian Paper Account

Alpaca can't trade Indian stocks, so this file *is* a broker — a complete simulated account stored as one JSON file.

7.1 nse_market_open (lines 34–40)

```
def nse_market_open(now=None):
    now = (now or datetime.now(timezone.utc)).astimezone(IST)   # IST = Asia/Kolkata
    if now.weekday() >= 5: return False              # Sat/Sun
    minutes = now.hour * 60 + now.minute
    return 9 * 60 + 15 <= minutes <= 15 * 60 + 30    # 09:15–15:30 IST
```

> **What/why:** Converts the current UTC time to Indian Standard Time and checks whether it's a weekday between 9:15 AM and 3:30 PM — NSE's trading session. The scanner calls this first and skips everything if the market is closed. It converts minutes-since-midnight for a clean range check. Holidays aren't modeled (a documented limitation) — on a holiday it may run but find no new signals, which is harmless.

7.2 The portfolio shape + buy (lines 45–116)

```
def _default(starting_cash):
    return {"currency": "INR", "starting_cash": starting_cash, "cash": starting_cash,
            "positions": {},        # symbol -> {qty, avg_price, opened_at}
            "trades": [],           # full trade log
            "equity_history": [], ...}  # snapshots for the chart

def buy(portfolio, symbol, price, notional=NOTIONAL):
    qty = int(min(notional, portfolio["cash"]) // price)     # whole shares within budget
    if qty <= 0 and MIN_ONE_SHARE and portfolio["cash"] >= price:
        qty = 1     # budget below share price → take 1 share
    if qty <= 0: return None
    cost = qty * price
    pos = portfolio["positions"].get(symbol, {"qty": 0, "avg_price": 0.0})
    if pos["qty"] == 0:
        pos["opened_at"] = _now()      # entry date for the time-exit rule
    total_qty = pos["qty"] + qty
    pos["avg_price"] = (pos["avg_price"] * pos["qty"] + cost) / total_qty   # weighted avg
    pos["qty"] = total_qty
    portfolio["cash"] -= cost
    portfolio["trades"].append({"ts": _now(), "symbol": symbol, "side": "buy", ...})
    return trade
```

> **What/why:** The portfolio is just a Python dict (saved as JSON). `buy` computes how many **whole shares** fit in the ₹1,000 budget (Indian markets have no fractional shares). The `MIN_ONE_SHARE` fallback handles the case where one share costs more than the budget — it buys a single share rather than skipping the stock entirely. When adding to an existing position it recomputes the **weighted average buy price**, which is what P&L is measured against. It records `opened_at` only on a fresh position — that timestamp drives the 7-day time-exit rule later. Cash is deducted, the trade is logged, and the trade is returned.

7.3 valuation — the numbers behind the dashboard (lines 131–168)

```
def valuation(portfolio, price_fn):
    for sym, pos in portfolio["positions"].items():
        price = price_fn(sym) or pos["avg_price"]     # live price, or fallback
        value = pos["qty"] * price
        rows.append({..., "pnl": round((price - pos["avg_price"]) * pos["qty"], 2), ...})
    realized = sum(t.get("pnl", 0) for t in trades if t.get("side") == "sell")
    earned = sum(r["pnl"] for r in rows if r["pnl"] > 0)   # winners
    lost   = sum(-r["pnl"] for r in rows if r["pnl"] < 0)  # losers
    return {"equity": cash + market_value, "total_return_pct": ...,
            "unrealized_earned": earned, "unrealized_lost": lost,
            "realized_pnl": realized, "positions": rows, ...}
```

> **What/why:** This "marks the portfolio to market" — for each holding it fetches the current price (via the passed-in `price_fn`, so the same logic works with any price source) and computes value and profit/loss versus the average buy price. It separates three kinds of P&L that the dashboard shows distinctly: **unrealized_earned** (paper gains on current winners), **unrealized_lost** (paper losses on current losers),

and **realized_pnl** (actual booked profit from stocks already sold, summed from the trade log). Passing `price_fn` as an argument is dependency injection — it keeps this pure-math function decoupled from how prices are actually fetched.

7.4 GitHub sync — the repo as a database (lines 196–215)

```
def gh_push(portfolio, message):
    url = f"https://api.github.com/repos/{repo}/contents/{PORTFOLIO_FILE}"
    r = requests.get(url, headers=headers)              # get current file SHA
    sha = r.json().get("sha") if r.status_code == 200 else None
    body = {"message": message,
            "content": base64.b64encode(json.dumps(portfolio).encode()).decode()}
    if sha: body["sha"] = sha                          # required to update existing file
    r = requests.put(url, headers=headers, json=body)
    return r.status_code in (200, 201)
```

What/why: This is the trick that lets the portfolio survive with your laptop off: it commits the JSON file directly to your GitHub repo via the Contents API. GitHub requires the current file's **SHA hash** to update it (so it can detect conflicting writes), so the code fetches that first, then base64-encodes the new content and PUTs it. Every trade thus becomes a git commit — a free, durable, fully-auditable database where the transaction history *is* the version history.

8. `alpaca.py` — The US Broker Client

8.1 The thin REST wrapper (lines 34–48)

```
def _request(method, path, base=BASE_URL, **kwargs):
    r = requests.request(method, base + path, headers=_headers(), timeout=20, **kwargs)
    if r.status_code == 404: return None
    r.raise_for_status()
    return r.json() if r.text else {}

def account(): return _request("GET", "/v2/account")
def clock():   return _request("GET", "/v2/clock")     # {is_open, next_open, ...}
def positions(): return _request("GET", "/v2/positions") or []
```

> **What/why:** Rather than pull in Alpaca's SDK, this is a hand-written client — just HTTP calls with the API keys in headers. `BASE_URL` defaults to the **paper** endpoint (`paper-api.alpaca.markets`) so no real money is ever at risk unless that env var is explicitly overridden. A 404 returns `None` (e.g. "no position in this symbol") rather than raising, which callers treat as "not held." Each function is a one-liner mapping to an Alpaca endpoint. Keeping it SDK-free means the cloud scanner installs almost nothing.

8.2 buy_notional — placing an order (lines 64–74)

```
def buy_notional(symbol, notional):
    stamp = datetime.now(timezone.utc).strftime("%Y%m%d%H%M%S")
    return _request("POST", "/v2/orders", json={
        "symbol": symbol.upper(), "notional": str(round(notional, 2)),
        "side": "buy", "type": "market", "time_in_force": "day",
        "client_order_id": f"{ORDER_PREFIX}-{symbol.upper()}-{stamp}"})
```

> **What/why:** Submits a market order for a fixed **dollar amount** (Alpaca supports fractional US shares, so "$1,000 of AAPL" works directly — unlike the Indian whole-share logic). The `client_order_id` is tagged with a `tvbot-` prefix and timestamp — this labels every order as placed by this bot, so it never gets confused with trades you make manually in Alpaca's own app. `time_in_force: "day"` means the order expires at end of day if unfilled (relevant when placed while the market is closed — it queues for the next open).

8.3 last_buy_fill_times — entry dates for the time-exit (lines 82–93)

```
def last_buy_fill_times(limit=200):
    orders = _request("GET", "/v2/orders",
                      params={"status": "closed", "limit": limit, "direction": "desc"})
    out = {}
    for o in orders:
        if o.get("side") == "buy" and o.get("filled_at"):
            sym = o["symbol"].upper()
            if sym not in out:            # first (most recent) wins
                out[sym] = datetime.fromisoformat(o["filled_at"].replace("Z", "+00:00"))
    return out
```

> **What/why:** Alpaca stores positions but not "when did I buy this." To power the 7-day time-exit rule, the scanner needs entry dates — so this fetches recent filled orders (newest first) and records the most recent buy fill time per symbol. Since the bot holds one position per symbol, the latest buy *is* the entry. If this call fails, the scanner degrades gracefully: stop-loss and take-profit still work; only the time-exit is skipped for that run.

9. `scanner.py` — The Autonomous Trader

This is the decision loop that runs every 30 minutes in the cloud. It never holds memory between runs — it re-derives everything from live flags + current positions.

9.1 The exit configuration (lines read from environment)

```
TAKE_PROFIT  = float(os.getenv("TV_BOT_TAKE_PROFIT", "0.05"))   # +5%
STOP_LOSS    = float(os.getenv("TV_BOT_STOP_LOSS", "0.04"))     # -4%
MAX_HOLD_DAYS = int(os.getenv("TV_BOT_MAX_HOLD_DAYS", "7"))      # 7 trading days
MIN_CONF     = float(os.getenv("TV_BOT_MIN_CONF", "0.55"))       # min confidence to buy
MAX_POS      = int(os.getenv("TV_BOT_MAX_POS", "10"))            # max open positions
```

> **What/why:** Every tunable number is read from an environment variable with a default. This means you can change the strategy's behaviour (e.g. take profit at 3% instead of 5%) by setting `TV_BOT_TAKE_PROFIT=0.03` — no code edit, no redeploy. Reading config from the environment is standard 12-factor-app practice; it also lets the GitHub Actions run and your laptop use different settings if you want.

9.2 exit_reason — the priority ladder (lines 30–52)

```
def exit_reason(pnl_frac, held_tdays, flag):
    if pnl_frac is not None:
        if STOP_LOSS > 0 and pnl_frac <= -STOP_LOSS:
            return "stop_loss"       # #1 — risk beats opinion
        if TAKE_PROFIT > 0 and pnl_frac >= TAKE_PROFIT:
            return "take_profit"     # #2 — bank the edge
    if flag == "SELL":
        return "sell_flag"          # #3 — engine turned bearish
    if MAX_HOLD_DAYS > 0 and held_tdays is not None and held_tdays >= MAX_HOLD_DAYS:
        return "time_exit"          # #4 — thesis expired
    return None                    # hold
```

> **What/why:** This tiny function encodes the entire short-swing exit policy, and the **order of the `if` statements is the priority**. Stop-loss is checked first — so a position down 4% is sold *even if the flag still says BUY* ("risk beats opinion"). Only if not stopped out does it check take-profit, then the SELL flag, then the 7-day time limit. Returning `None` means "no exit condition met → keep holding." Pure function, no side effects — which is why it's trivial to unit-test (as shown in the verification section earlier).

9.3 _trading_days_between (lines 18–28)

```
def _trading_days_between(start, end):
    if start is None or end <= start: return 0
    days, cur = 0, start.date()
    while cur < end.date():
        cur += timedelta(days=1)
        if cur.weekday() < 5: days += 1     # Mon–Fri only
    return days
```

> **What/why:** The time-exit counts *trading* days, not calendar days — a position bought Friday isn't "3 days old" by Monday just because a weekend passed. This walks day by day and counts only weekdays. Holidays aren't modeled (a known simplification), so a holiday just makes a "7-day" hold exit one scan later — harmless.

9.4 The reconcile loop — the heart of run_virtual_scan (NSE)

```
if pos:
    pnl_frac = (price / pos["avg_price"] - 1) if pos.get("avg_price") else None
    opened = pos.get("opened_at")
    if not opened:                          # position predates exit tracking
        pos["opened_at"] = datetime.now(timezone.utc).isoformat()
        held_days = 0
    else:
        held_days = _trading_days_between(datetime.fromisoformat(opened), datetime.now(timezone.utc))
    reason = exit_reason(pnl_frac, held_days, flag)
    if reason:
        trade = vb.sell_all(p, sym, price, reason=reason)
        say(f"    -> SOLD {trade['qty']} @ ₹{price} ({reason}, pnl ₹{trade['pnl']:,.0f})")
    else:
        say(f"    -> holding (pnl {pnl_frac*100:+.1f}%, day {held_days}/{MAX_HOLD_DAYS})")
elif flag == "BUY":
    if conf < MIN_CONF: say("skip buy (confidence too low)")
    elif open_count >= MAX_POS: say("skip buy (max positions)")
    else: trade = vb.buy(p, sym, price); ...
```

> **What/why:** This is the core logic. For each watchlist symbol: **if we already hold it**, compute PnL fraction and days held, ask `exit_reason` what to do, and either sell (logging *which* rule fired) or print the "holding" line. **If we don't hold it and the flag is BUY**, check the confidence floor and position cap, then buy. The `if not opened` branch handles positions bought before the exit feature existed — it starts their clock now instead of falsely treating them as ancient. This is the "reconciliation" pattern: compare desired state (flags) with actual state (positions) and act on the difference.

9.5 Guardrails before the loop even runs (Alpaca/US scan)

```
clk = alpaca.clock()
if not clk.get("is_open"):
    say(f"Market closed (next open {clk.get('next_open')}) — nothing to do.")
    return {"status": "market_closed", ...}
acct = alpaca.account()
equity, last_equity = float(acct["equity"]), float(acct["last_equity"] or equity)
if last_equity > 0 and equity < last_equity * (1 - MAX_DD):
    say(f"CIRCUIT BREAKER: equity down ... Turning AUTO_TRADING off.")
    set_auto_trading(False)                 # the bot disables itself
    return {"status": "circuit_breaker", ...}
```

> **What/why:** Before touching any position, two safety gates. First, the market-clock check — trading when closed is pointless, so it exits early. Second, the **drawdown circuit breaker**: if the account's equity has dropped more than 3% (`MAX_DD`) since yesterday's close, the bot **flips its own AUTO_TRADING switch off** and stops — a fail-safe so a bad day can't cascade. That `set_auto_trading(False)` call reaches out to the GitHub API to change the repo variable, which means even the cloud scans stop until you manually re-enable.

9.6 The GitHub variable read/write — get/set_auto_trading (lines 30–75)

```
def get_auto_trading():
    r = requests.get(f"https://api.github.com/repos/{repo}/actions/variables/AUTO_TRADING",
                     headers=headers)
    if r.status_code == 404: return False       # variable not created yet
    return r.json().get("value", "").strip().lower() == "on"
```

> **What/why:** The ON/OFF switch is a GitHub repo variable, readable/writable via GitHub's REST API. This is what lets the sidebar toggle (running on your laptop) control whether the cloud scanner trades (running on GitHub's servers) — they share one source of truth that lives on GitHub, reachable from anywhere. A 404 (variable never created) is treated as OFF, the safe default.

10. `server.py` — The Web API

The FastAPI server is the bridge between the browser sidebar and all the Python logic. Every button and question in the sidebar becomes an HTTP call here.

10.1 Cached analysis + auto-journaling (lines 72–87)

```
def _get_analysis(symbol):
    key = symbol.upper().strip()
    with _cache_lock:
        hit = _cache.get(key)
        if hit and now - hit["ts"] < CACHE_TTL_SECONDS:   # 300s = 5 min
            return hit["analysis"]
    analysis = analyze_ticker(key)
    if "error" not in analysis:              # never cache failures
        with _cache_lock:
            _cache[key] = {"ts": now, "analysis": analysis}
        try: journal.log_flag(analysis)        # record for the scoreboard
        except Exception: pass                 # journaling must never break analysis
    return analysis
```

What/why: Every symbol's analysis is cached for 5 minutes, so rapidly re-opening a chart or asking several questions doesn't re-run the whole 17-strategy engine each time. The `_cache_lock` makes this thread-safe (FastAPI can handle requests concurrently). Two design choices stand out: **failures are never cached** (a rate-limit error is transient, so caching it would lock in a bad result), and every successful analysis is **logged to the journal** for later outcome-scoring — wrapped in try/except so a journaling bug can never break the actual analysis response.

10.2 The Gemini call with fallback ladder (lines 148–196)

```
for model in GEMINI_MODELS:            # [flash-latest, flash-lite-latest, 2.0-flash]
    r = requests.post(GEMINI_URL.format(model=model), params={"key": key}, json=payload)
    if r.status_code in (429, 503):      # quota exhausted / overloaded
        continue                         # → try the next model
    r.raise_for_status()
    answer = "".join(p.get("text", "") for p in parts).strip()
    if answer: break
```

What/why: The chat sends the system prompt + compacted evidence JSON + last 8 conversation turns to Gemini. The `for` loop is the **reliability ladder**: if the preferred model is rate-limited (429) or overloaded (503), it silently tries the next model instead of failing. If all models fail, `_llm_answer` returns `None`, and the caller falls back to a rule-based text summary — so the chat box never goes fully dead. This layered fallback is why the product works on a free tier where any single model can be temporarily unavailable.

10.3 papertrade — routing US vs India (lines 253–293)

```
def papertrade(req: TradeRequest):
    analysis = _get_analysis(req.symbol)
    yf_sym = (analysis.get("yf_symbol") or "").upper()
    if yf_sym.endswith(".NS") or yf_sym.endswith(".BO"):
        return _indian_papertrade(analysis, req.side)     # virtual broker
    # else: Alpaca path
    act = _effective_side(analysis, req.side)
    if act == "buy":
        order = alpaca.buy_notional(sym, notional)
        return {"ok": True, "action": f"BUY ${notional} of {sym}", ...}
```

What/why: One endpoint, two markets. It looks at the resolved Yahoo symbol: if it ends in `.NS` / `.BO` it's Indian → route to the virtual broker; otherwise it's US → route to Alpaca. `_effective_side` resolves what "buy/sell" means: an explicit button press wins, otherwise it follows the current flag. This is why the same "📄 Buy" button works correctly on both an Indian and a US stock without the sidebar needing to know the difference.

10.4 The performance page assembly (lines 649–659)

```
def performance():
    try: journal.update_outcomes()          # fill in 5/10/20-day results
    except Exception: pass
    html = journal.render_html().replace("</body>", _portfolio_html() + "</body>")
    html = html.replace("</h1>", "</h1>" + _NAV, 1)
    return HTMLResponse(html)
```

> **What/why:** The `/performance` page is built by string-composition: the journal renders the flag scoreboard, then the portfolio HTML (chart + holdings table) is injected before `</body>`, and the nav bar after the first `</h1>`. Before rendering it calls `update_outcomes()`, which fetches what price did 5/10/20 days after each past flag — turning "the bot said BUY" into "and it was right/wrong." This is the accountability layer that makes the whole project measurable rather than just plausible.

11. `watchlists.py` — Managing the Scanner's Universe

11.1 classify — routing a symbol to a market (lines 30–39)

```
def classify(symbol):
    yf_sym = tradingview_to_yfinance(symbol.strip().upper())
    if yf_sym.endswith(".NS") or yf_sym.endswith(".BO"):
        return "in", yf_sym                    # Indian → watchlist_in.txt
    if yf_sym and all(c not in yf_sym for c in ".-="):
        return "us", yf_sym                    # plain ticker → watchlist.txt
    return None, yf_sym                       # crypto/FX → unsupported
```

> **What/why:** When you click the ★ to add a stock to the scanner, this decides which file it belongs in. Indian symbols (with a `.NS` / `.BO` suffix) go to the NSE watchlist; clean tickers with no punctuation (AAPL, NVDA) go to the US watchlist; anything with a dash or equals (crypto `BTC-USD`, forex `EUR=X`) returns `None` — unsupported, because neither broker trades them. The market detection is automatic, so you never pick a list manually.

11.2 toggle — add/remove + GitHub sync (lines 96–120)

```
def toggle(symbol):
    market, yf_sym = classify(symbol)
    if market is None: return {"error": "...US stocks or NSE only"}
    current = _gh_get(market) or _read(FILES[market])   # prefer repo copy
    if yf_sym in current:
        current = [s for s in current if s != yf_sym]      # remove
        watching = False
    else:
        current.append(yf_sym)                          # add
        watching = True
    _write(market, current)                             # local file
    synced = _gh_put(market, current, action)           # push to repo
    return {"ok": True, "watching": watching, "synced_to_github": synced, ...}
```

> **What/why:** The ★ button is a toggle: if the symbol is already in the list, remove it; otherwise add it. Critically, it reads the **GitHub copy first** (the cloud scanner's source of truth) so it doesn't clobber changes made elsewhere, writes the local file, then pushes back to GitHub. This is why adding a stock via the sidebar immediately affects what the cloud scanner trades on its next run — the watchlist files in the repo are the scanner's entire universe.

12. `content.js` & `background.js` — The Chrome Extension

12.1 background.js — the network proxy (whole file)

```
const API = "http://127.0.0.1:8765";
chrome.runtime.onMessage.addListener((msg, _sender, sendResponse) => {
  const opts = { method: msg.method || "GET" };
  if (msg.body) {
    opts.method = "POST";
    opts.headers = { "Content-Type": "application/json" };
    opts.body = JSON.stringify(msg.body);
  }
  fetch(API + msg.path, opts)
    .then((r) => r.json())
    .then((data) => sendResponse({ ok: true, data }))
    .catch((e) => sendResponse({ ok: false, error: String(e) }));
  return true;   // keep the channel open for the async response
});
```

What/why: This tiny service worker exists to solve one specific problem: a script running inside the `https://tradingview.com` page is blocked by browser security (CSP + private-network rules) from calling `http://127.0.0.1`. But the extension's *background worker* is allowed to, because `manifest.json` grants it `host_permissions` for that address. So the sidebar sends a message here, the worker makes the actual fetch, and sends the result back. The `return true` is a required Chrome quirk — it tells Chrome the response will come asynchronously, keeping the message channel open.

12.2 detectSymbol — knowing which stock you're viewing (lines 23–41)

```
function detectSymbol() {
  const u = new URL(window.location.href);
  const q = u.searchParams.get("symbol");
  if (q) return decodeURIComponent(q);              // /chart/?symbol=NSE:RELIANCE
  const sm = u.pathname.match(/^\/symbols\/([A-Z0-9.]+)-([A-Z0-9.!&_]+)\/?/);
  if (sm) return `${sm[1]}:${sm[2]}`;                // /symbols/NASDAQ-AAPL/
  if (!u.pathname.startsWith("/chart")) return null;
  const m = (document.title||"").match(/^([A-Z0-9.\-:_!&]{1,20})[\s,]/);
  return m ? m[1] : null;
}
```

What/why: TradingView is a single-page app — the URL and title change as you navigate without a full page reload. This function figures out the current symbol from three sources in order: the `?symbol=` URL parameter, the `/symbols/EXCHANGE-TICKER/` path pattern (converted back to `EXCHANGE:TICKER`), or the page title as a last resort. The `if (!u.pathname.startsWith("/chart"))` guard prevents mistaking a screener/list page title for a ticker. This detection is what makes the sidebar "follow" you as you browse.

12.3 The polling loop that ties it together (lines 242–251)

```
setInterval(() => {
  const s = detectSymbol();
  if (s && s !== currentSymbol) {          // symbol changed?
    currentSymbol = s;
    $("tvb-chat").innerHTML = "";
    addMsg(`Now watching ${s}...`, "bot");
    refreshAnalysis(s);                    // GET /analyze → paint the flag
    refreshWatch(s);                       // GET /watchlist/status → paint the ★
  }
}, 1500);
```

What/why: Every 1.5 seconds the sidebar checks if you've navigated to a different stock. When the symbol changes, it clears the chat, fetches a fresh analysis (which paints the BUY/SELL/ HOLD badge and reasons), and updates the ★ watchlist star. Polling is used because TradingView doesn't emit an event when the symbol changes — 1.5s is frequent enough to feel instant but light enough not to hammer the server (and the 5-min server cache absorbs repeats). This loop is the "engine" of the sidebar UX.

13. The Life of One Trade — Full Worked Example

Now let's trace **one real scenario** end-to-end, naming the exact file, function, and value change at each step. Scenario: you open **NSE:RELIANCE**, the bot flags BUY, you add it to the watchlist, the scanner buys it, holds for several days, and finally sells at a profit.

Step 1 — You open NSE:RELIANCE on TradingView

`content.js` · `detectSymbol()` reads the URL, returns `"NSE:RELIANCE"`. The polling loop sees this differs from `currentSymbol`, so it calls `refreshAnalysis("NSE:RELIANCE")`, which sends a message to `background.js`, which does `fetch("http://127.0.0.1:8765/analyze?symbol=NSE:RELIANCE")`.

Step 2 — The server receives /analyze

`server.py` · `analyze()` → `_get_analysis("NSE:RELIANCE")`. Cache miss (first view), so it calls `analyze_ticker("NSE:RELIANCE")`.

Step 3 — Symbol resolution & data fetch

`data.py` · `resolve_symbol()` maps `NSE:RELIANCE` → `RELIANCE.NS` (via `EXCHANGE_SUFFIX["NSE"] = ".NS"`), confirms it has data. `fetch_frames()` returns 4 DataFrames: `1W` (261 weekly bars), `1D` (500 daily bars), `1H` / `4H` (intraday). Say the current price comes back as **₹1,327**.

Step 4 — Strategies run and vote

`engine.py` · `_analyze_frame("1D", daily_df)` runs all 13 detectors. Suppose on the daily frame: `classify_trend()` returns `"up"` → `market_structure` Signal(dir=+1, strength=0.8, weight=1.5×1.0). `trend_following_signal()` returns bullish golden-cross → Signal(+1, 0.85, 1.2). `detect_break()` finds a bullish BOS → Signal(+1, 0.7, 1.2). But `divergence` on the weekly finds a bearish RSI divergence → Signal(−1, 0.7, 1.3×1.3). The weekly `market_structure` is also up → Signal(+1, 0.8, 1.5×1.3).

Step 5 — Aggregation into a flag

`scoring.py` · `aggregate()` computes `score = Σ(direction×strength×weight) / Σweight`. Suppose the bullish signals outweigh the one bearish divergence: **score = +0.28**. Since `0.28 ≥ BUY_THRESHOLD (0.22)` → **flag = "BUY"**. Agreement is fairly one-sided → **confidence = 0.62**. The top reasons list is built: `["[1W] structure is up (HH/HL)", "[1D] golden cross...", ...]`.

Step 6 — The sidebar paints the result

`content.js` · `refreshAnalysis()` receives the JSON, sets the flag badge to green **"BUY"** (`FLAG_COLORS.BUY = "#16a34a"`), shows "score +0.28 · confidence 62% · 6▲ / 2▼ signals", and lists the reasons. You click the ★ → `/watchlist/toggle` → `watchlists.py` `classify()` returns `("in", "RELIANCE.NS")`, appends it to `watchlist_in.txt`, and pushes to GitHub. RELIANCE is now in the scanner's universe.

Step 7 — The scanner buys (next NSE-hours run)

`scanner.py` · `run_virtual_scan()`. Market open ✓. RELIANCE flag is still BUY, confidence 0.62 ≥ `MIN_CONF (0.55)` ✓, not already held ✓, under 10 positions ✓. It calls `virtual_broker.py` `buy(p, "RELIANCE.NS", 1327.0)`: `qty = int(min(1000, cash) // 1327)` = 0, but the `MIN_ONE_SHARE` fallback buys **1 share** for ₹1,327. `opened_at` is stamped with today's date. Log: `-> BOUGHT 1 @ ₹1327`. The portfolio JSON is committed to GitHub.

Step 8 — Holding across days

Each subsequent scan: RELIANCE is now held, so the `if pos:` branch runs. `pnl_frac = price/1327 − 1`. Day 2 price ₹1,340 → pnl +1.0%, held 1 day → `exit_reason(0.01, 1, flag)` returns `None` → log `-> holding (pnl +1.0%, day 1/7)`. Day 4 price ₹1,320 → pnl −0.5%, still no exit → holds.

Step 9 — Take-profit fires

Day 6, RELIANCE rallies to **₹1,395**. `pnl_frac = 1395/1327 − 1 = +0.051` (+5.1%). `exit_reason(0.051, 5, "HOLD")`: stop-loss? no. take-profit? `0.051 ≥ 0.05` → **returns "take_profit"**. The scanner calls `sell_all(p, "RELIANCE.NS", 1395.0,`

`reason="take_profit")` : cash += ₹1,395, position removed, trade logged with `pnl = (1395−1327)×1 = ₹68` and `reason="take_profit"` . Log: `-> SOLD 1 @ ₹1395 (take_profit, pnl ₹68)` .

Step 10 — It shows up on the dashboard

`server.py` · `/performance` → `virtual_broker.py` `valuation()` now counts that ₹68 in `realized_pnl` , the equity curve ticks up, and the trade appears in the log with its `take_profit` reason. The whole loop — browse → flag → watchlist → buy → hold → profit-take → dashboard — completed with **zero manual intervention** after Step 6, running in the GitHub cloud even with your laptop off.

The one-sentence summary of the whole system: `data.py` fetches candles → the `technicals/` strategies each vote → `scoring.py` turns votes into a flag → `engine.py` orchestrates it → `scanner.py` acts on the flag with risk guardrails → `virtual_broker.py` / `alpaca.py` hold the money → `server.py` serves it to the `chrome_extension/` sidebar → GitHub Actions runs the whole thing on a schedule. Every number you see traces back to a specific line that computed it.